

Recursive Proof Aggregation with deferred pairing check

Contents

1	Overview	1
2	Decomposed accumulators	4
3	A PCD-style accumulation scheme induced by $(\text{Combine}, D_{\mathcal{B}})$	6
4	Underlying proof system and proof-accumulators	8
5	PCD-style recursion: relations, predicate, and verifier	9
5.1	Leaf relation (layer $\ell = 0$)	9
5.2	Aggregation relations for $\ell \geq 1$	9
6	Completeness	11
7	Knowledge soundness	14
8	Remark on “fixed-base MSM deferred to the root”	16
9	Rollup state transitions	16
9.1	Context and high-level goal	16
9.2	Merkle-map interface (MapGadget semantics)	17
9.3	Client bundles and their public effects	18
9.4	Bundle-induced rollup state transition	20
9.5	Aggregation state, instances, and messages	21
9.6	Local compliance predicate φ_{ROLLUP}	21
9.7	Historic-root update and terminal predicate ψ_{ROLLUP}	23

Lydia: Not important but once we are finished, maybe this could be relabeled preliminaries and a more high level overview could be added that doesn't include mathematical notation. Probably best to wait until it is finished to avoid rewrites. At the end it would be nice to include some more intuition so that anyone, even someone not so familiar with recursive proving but a little cryptography knowledge could follow this document for when it becomes a white paper.

1 Overview

This document specifies and proves correctness properties of a binary recursive proof aggregator with a *deferred pairing check*. The presentation is organized so that the recursion and the proofs match the accumulation-based PCD framework of [BCMS, §5.1/5.3/5.4], while remaining faithful to the concrete behavior of our implementation (`AggCircuit`).

The recursive proof aggregation at a high-level. We organize proofs in a binary recursion tree. Each node at layer ℓ produces a proof for a *public instance* x_ℓ and consumes two child proofs from layer $\ell - 1$. *Lydia: maybe we could label x by left or right?*

Messages. For recursion we pass upward a compact *message* that represents the child node’s public claim in compressed form. Concretely, a message at layer ℓ is

$$z_\ell := (s_\ell, \ell),$$

Lydia: maybe we could label z as well by left or right? where $s_\ell \in \mathbb{F}$ is a digest of the child’s full public instance x_ℓ (defined below) and ℓ is an explicit layer tag. The digest is computed by hashing the public data deterministically (e.g., with Poseidon):

$$s_\ell := \text{Hash}(\text{encode}(x_\ell)).$$

At layer 0, x_0 is the client’s public input (or its encoding), so s_0 is exactly the client instance digest.

Compression via Merkle-style hashing. To bind an entire batch (the leaves under a node) without carrying all leaf public inputs, the recursion maintains a Merkle-style hash over child digests: at an internal node,

$$s_\ell := \text{Hash}(s_{\ell-1}^L, s_{\ell-1}^R).$$

Lydia: maybe we could use a different notation here to make it clear whether s_ℓ refers to the merkle style hash or $(\text{encode}(x_\ell))$ Equivalently, s_ℓ is the root of a binary hash tree whose leaves are the public-input digests of the aggregated proofs (and whose internal nodes are Poseidon hashes). This makes the aggregated proof commit to the *exact multiset and order* of the included leaf instances while keeping the public instance size constant.

Within each recursive step, the circuit enforces two conceptually separate components:

1. **Local compliance (composition correctness).** Given two child instances $x_{\ell-1}^L, x_{\ell-1}^R$ and their messages $z_{\ell-1}^L = (s_{\ell-1}^L, \ell - 1)$, $z_{\ell-1}^R = (s_{\ell-1}^R, \ell - 1)$, the circuit checks that the parent message $z_\ell = (s_\ell, \ell)$ is consistent with them, namely that $s_\ell = \text{Hash}(s_{\ell-1}^L, s_{\ell-1}^R)$, that layer tags match, and that the “wiring constraints” between the two children hold (e.g., root-stitching or boundary-equality constraints for the rollup state).
2. **Deferred verification via accumulation (pairing deferred to a decider).** Instead of performing the final pairing check of the underlying PLONK/KZG verifier at every recursion step, the circuit maintains a compact *accumulator* representing the verifier’s remaining pairing equation. This relies on two facts: KZG verification equations are linear (homomorphic) in the committed data, and the pairing is bilinear. As a result, multiple KZG opening checks can be *batched* into a single randomized linear combination: each recursive step extracts two new proof-induced pairing contributions from the child proofs and folds them together with two previously carried accumulators into one new accumulator using transcript-derived random weights. The expensive operations (evaluating the fixed-base MSM and executing the pairing) are performed only once by a terminal *decider*, which checks a single pairing equation.

Thus, verifying an aggregated proof consists of: (i) verifying the step proof (which enforces local compliance and correct accumulator folding), and (ii) running the terminal decider to check that the final carried accumulator satisfies the pairing equation.

Underlying proof systems and partial verification. For each layer $\ell \geq 0$, let ARG_ℓ be the proof system used to prove satisfiability of a relation R_ℓ , with verification predicate Verify_ℓ (Section 3). *Lydia:could you give the syntax for ARG_ℓ* We also define a deterministic *partial verifier* map

$$\text{PA}_\ell : \mathsf{X}_\ell \times \Pi_\ell \rightarrow \text{DA}_\mathcal{B} \times \{0, 1\},$$

which runs the layer- ℓ verifier up to (but excluding) the final pairing check and outputs a *proof-item* $\rho \in \text{DA}_\mathcal{B}$ together with a bit b indicating whether all non-pairing checks passed (Section 5, “Underlying proof system and proof-accumulators”). When $b = 1$, acceptance of the full verifier is characterized by a single pairing equation on ρ , captured by the decider $\text{D}_\mathcal{B}$ (Definition 2.3). *Lydia:Could you briefly introduce $\mathsf{X}_\ell, \Pi_\ell, \text{DA}_\mathcal{B}$*

Accumulators and folding. *Lydia:I’m wondering if given a lot of definitions are in this section, it might make sense to include the accumulator definition?* Accumulators are represented as *decomposed* MSM expressions (variable-base terms plus a fixed-base scalar map over a global basis set $\mathcal{B}$); evaluating them yields the two group elements in the pairing equation checked by $\text{D}_\mathcal{B}$ (Section 2). We fold multiple accumulators into one using a deterministic combining operator **Combine** (Definition 2.5), and package this as a concrete accumulation scheme **AS** (Definition 3.1), whose verification step simply recomputes **Combine** and checks equality. The expensive work (the pairing) is performed by the decider $\text{D}_\mathcal{B}$.

Concrete recursive statement format. Leaves (layer 0) prove a base relation R_0 where the public instance is a single digest $s \in \mathbb{F}$ of a larger vector (Section 5). For each layer $\ell \geq 1$, the public instance is

$$x_\ell = (s, \text{acc}, \ell),$$

where $s \in \mathbb{F}$ is the node state/digest, $\text{acc} \in \text{DA}_\mathcal{B}$ is the carried accumulator (encoded via an injective $\text{Pl}(\text{acc})$ as public inputs), and ℓ is a layer tag. *Lydia:Do we need left and right here too*

What the step circuit enforces. For $\ell \geq 1$, the circuit for layer ℓ checks:

1. a *local compliance predicate* φ stating (a) $s = \text{Hash}(s_L, s_R)$, (b) each child proof passes the partial-verifier relation $R_{\ell-1}^{\text{PA}}$ producing items ρ_L, ρ_R , and (c) incoming messages match the parsed child digests (Definition 5.1);
2. an AS-verification step that folds $[\rho_L, \rho_R]$ together with two “old” carried accumulators into the parent accumulator acc (neutralizing carried inputs at $\ell = 1$).

Correctness and security claims. *Lydia:A little more detail on what these claims are could be added here and a little more intuition included* We prove perfect completeness from (i) perfect completeness of ARG_ℓ for all layers and (ii) completeness of **AS** (Theorem 6.1). We prove knowledge soundness via the same level-by-level extractor strategy as [BCMS], using (i) knowledge soundness of ARG_ℓ and (ii) soundness of the folding operator (**Combine**, $\text{D}_\mathcal{B}$) packaged as **AS** (Theorem 7.1).

Instantiation for rollups. In Section 7 we instantiate the abstract digest s and local predicate φ with a rollup state machine: s commits to the ordered leaf instances and the before/after map roots, and φ enforces correct Merkle-map updates and recursion stitching.

2 Decomposed accumulators

Let $\mathbb{F}$ be a finite field of prime order p and $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ finite groups, also of order p . Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be an efficiently computable, non-degenerate pairing. We denote by $g_1 \in \mathbb{G}_1, g_2 \in \mathbb{G}_2$ generators and define $g_T := e(g_1, g_2) \in \mathbb{G}_T$. Note that, since e is non-degenerate, g_T also generates $\mathbb{G}_T$. We write $\mathbb{G}_1, \mathbb{G}_2$ additively and $\mathbb{G}_T$ multiplicatively.

For $x \in \mathbb{F}$, define $[x]_1 := x \cdot g_1$, $[x]_2 := x \cdot g_2$, and $[x]_T := g_T^x$. With this convention, $e([a]_1, [b]_2) = [ab]_T$ for all $a, b \in \mathbb{F}$. Let $d \in \mathbb{N}$ and

$$\left([1]_1, [\tau]_1, \dots, [\tau^{2^d-1}]_1, [1]_2, [\tau]_2 \right)$$

be a KZG SRS for some $\tau \in \mathbb{F}$.

Definition 2.1 (Negligible function). *A function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is negligible if for every polynomial $q(\cdot)$ there exists N such that for all $\lambda \geq N$, $\text{negl}(\lambda) < 1/q(\lambda)$.*

Remark 2.1 (Security parameter and field size). *We view the prime order $p = |\mathbb{F}|$ as a function of the security parameter λ and assume p is super-polynomial (typically exponential) in λ , as in standard pairing-friendly curve instantiations.*

Motivation: Global fixed bases. Our PLONK partial verifier uses a substantial number of *fixed* curve points (e.g., selector commitments, verification-key elements, and other circuit-constant bases). Consequently, a large portion of the accumulator carried by the verifier is naturally expressed as a *fixed-base MSM*. Rather than fully evaluating this fixed-base MSM at every recursive step, we *delay its evaluation* until the final stage when the terminal accumulator is opened: across intermediate recursion layers we only *combine the associated scalars* for the fixed bases, and defer the actual scalar multiplication and summation of fixed points.

In our recursion, verifier layers are arranged as a binary tree, and different layers may use different sets of fixed bases. To obtain a uniform interface, we introduce a *global* fixed-base set $\mathcal{B}$ defined as the union of all fixed bases appearing in any layer. At a given layer, scalars corresponding to bases not used by that layer are set to zero (equivalently, omitted and treated as default-0), so that every layer manipulates fixed-base contributions in the same ambient index set. [Lydia: I'm not familiar with the meaning of ambient in this context](#)

Crucially, the final KZG verification performed by the PLONK prover reduces to a single pairing check, and this check can be expressed exactly in the accumulator form defined below. The role of the recursive verifier is therefore to maintain and combine such accumulators, postponing the evaluation of fixed-base MSMs until this terminal pairing check.

Global fixed bases set. Let Name be a *finite* set of identifiers and let $\mathcal{B} = \{B_\beta\}_{\beta \in \text{Name}} \subseteq \mathbb{G}_1$ be the *global union* of all fixed bases used across all verifier gadgets / circuits. [Lydia: I think maybe remove the verifier gadgets as this hasn't been mentioned yet](#) $\mathcal{B}$ will be fixed for the remainder of this article. We assume *name consistency*, namely that each $\beta \in \text{Name}$ uniquely identifies a point $B_\beta \in \mathcal{B}$.

Definition 2.2 (Decomposed MSM). *A decomposed MSM is a pair $\text{msm} = (\text{var}, \text{fix})$ where: (i) var is a multiset of variable-base terms $\{(P_i, s_i)\}_{i=1}^t$ with $P_i \in \mathbb{G}_1$ and $s_i \in \mathbb{F}$; (ii) $\text{fix} : \text{Name} \rightarrow \mathbb{F}$ is a fixed-base scalar map (missing keys default to 0). Define evaluation w.r.t. $\mathcal{B}$ by*

$$\text{Eval}_{\mathcal{B}}(\text{msm}) := \sum_{i=1}^t s_i P_i + \sum_{\beta \in \text{Name}} \text{fix}(\beta) B_\beta \in \mathbb{G}_1.$$

It will be useful to have the notion of a *zero decomposed MSM* $\text{ZeroMSM} := (\text{var}, \text{fix})$, where $\text{var} = \emptyset$ and $\text{fix}(\beta) = 0$, for all $\beta \in \text{Name}$.

Definition 2.3 (Decomposed accumulator and decider). A decomposed accumulator $\text{acc} = (\text{lhs}, \text{rhs})$ is a pair of decomposed MSMs. We use $\text{DA}_{\mathcal{B}}$ to denote the set of decomposed accumulators. *Lydia: Does the syntax for a decomposed accumulator differ from the syntax for an accumulator* Define the decider $D_{\mathcal{B}} : \text{DA}_{\mathcal{B}} \rightarrow \{0, 1\}$, by

$$D_{\mathcal{B}}(\text{acc}) = 1 \iff e(\text{Eval}_{\mathcal{B}}(\text{lhs}), [\tau]_2) = e(\text{Eval}_{\mathcal{B}}(\text{rhs}), [1]_2),$$

for all $\text{acc} \in \text{DA}_{\mathcal{B}}$. This ultimately arises from the final KZG pairing check in the PLONK verifier ([GWC, §8.3]). *Lydia: Mention where τ comes from*

Again, we define the notion of a *zero decomposed accumulator* $\text{ZeroAcc} := (\text{ZeroMSM}, \text{ZeroMSM})$. Note that, trivially, we have $D_{\mathcal{B}}(\text{ZeroAcc}) = 1$.

Definition 2.4 (Linear combinations of decomposed accumulators). Given $r \in \mathbb{F}$ and $\text{acc} = (\text{lhs}, \text{rhs}) \in \text{DA}_{\mathcal{B}}$, we define $r \cdot \text{acc} \in \text{DA}_{\mathcal{B}}$ to be obtained by scaling all the scalars corresponding to both the variable-bases and fixed-bases in both lhs and rhs by r .

Given decomposed accumulators $\text{acc}_1 = (\text{lhs}_1, \text{rhs}_1)$, $\text{acc}_2 = (\text{lhs}_2, \text{rhs}_2)$, we define

$$\text{acc}_1 + \text{acc}_2 = \text{acc} := (\text{lhs}, \text{rhs}) \in \text{DA}_{\mathcal{B}},$$

where lhs (resp. rhs) is obtained by concatenating the variable-base terms and summing the fixed-base scalars in lhs_1 and lhs_2 (resp. rhs_1 and rhs_2).

Combining these two definitions, we can now define taking linear combinations of decomposed accumulators in the obvious way.

Public input encoding. For $\text{acc} = (\text{lhs}, \text{rhs}) \in \text{DA}_{\mathcal{B}}$, let $\text{PI}(\text{acc}) \in \mathbb{F}^*$ be a fixed deterministic and *injective* encoding of acc (w.r.t. $(\text{Name}, \mathcal{B})$), used as public inputs. Concretely, PI must fix an ordering of Name and serialize (lhs, rhs) with all fixed-base scalars listed in that order (missing keys treated as 0).

Lydia: Maybe start with some syntax here or at the beginning of this section, there is no definition for the accumulator combination before it is instantiated. Also move the definitions from Lemma 2.1 to the top.

Definition 2.5 (Accumulator combination operator). Let $H : \mathbb{F}^* \rightarrow \mathbb{F}$ be a hash (modeled as a random oracle). Given $(\text{acc}_1, \dots, \text{acc}_n) \in (\text{DA}_{\mathcal{B}})^n$, define $\text{Combine}(\text{acc}_1, \dots, \text{acc}_n) \in \text{DA}_{\mathcal{B}}$. *Lydia: I find the below algorithm below hard to parse, I think because it is split between the list and the final computation* *Lydia: mention PI*

1. $r \leftarrow H(\text{PI}(\text{acc}_1) \parallel \dots \parallel \text{PI}(\text{acc}_n));$
2. weights $r^0, r^1, \dots, r^{n-1};$
3. output the decomposed accumulator obtained by the linear combination:

$$\text{Combine}(\text{acc}_1, \dots, \text{acc}_n) := \sum_{i=1}^n r^{i-1} \cdot \text{acc}_i.$$

Lemma 2.1 (Accumulator invariant under combination). The pair $(\text{Combine}, D_{\mathcal{B}})$ satisfies:

1. *Perfect completeness:* *Lydia:* For $n \in \mathbb{N}$, for any $\{\text{acc}_i\}_{i \in [n]}$ such that $\text{acc}_i \in \text{DA}_{\mathcal{B}}$ if $D_{\mathcal{B}}(\text{acc}_i) = 1$ for all i , then $D_{\mathcal{B}}(\text{Combine}(\text{acc}_1, \dots, \text{acc}_n)) = 1$.
2. *Soundness:* *Lydia:* For $n \in \mathbb{N}$, for any $\{\text{acc}_i\}_{i \in [n]}$ such that $\text{acc}_i \in \text{DA}_{\mathcal{B}}$ if $D_{\mathcal{B}}(\text{Combine}(\text{acc}_1, \dots, \text{acc}_n)) = 1$, then $D_{\mathcal{B}}(\text{acc}_i) = 1$ for all i , except with probability $\text{negl}(\lambda)$.

Proof. For each i , write $\text{acc}_i = (\text{lhs}_i, \text{rhs}_i)$ and abbreviate $L_i := \text{Eval}_{\mathcal{B}}(\text{lhs}_i) \in \mathbb{G}_1$, $R_i := \text{Eval}_{\mathcal{B}}(\text{rhs}_i) \in \mathbb{G}_1$. Define the error ratio

$$S_i := \frac{e(L_i, [\tau]_2)}{e(R_i, [1]_2)} \in \mathbb{G}_T.$$

Then $D_{\mathcal{B}}(\text{acc}_i) = 1 \iff S_i = 1$. Let $r := H(\text{PI}(\text{acc}_1) \parallel \dots \parallel \text{PI}(\text{acc}_n)) \in \mathbb{F}_p$ and $\text{acc}^* := \text{Combine}(\text{acc}_1, \dots, \text{acc}_n)$. Linearity gives $\text{Eval}_{\mathcal{B}}(\text{lhs}^*) = \sum_{i=1}^n r^{i-1} L_i$ and $\text{Eval}_{\mathcal{B}}(\text{rhs}^*) = \sum_{i=1}^n r^{i-1} R_i$. By bilinearity,

$$D_{\mathcal{B}}(\text{acc}^*) = 1 \iff \prod_{i=1}^n S_i^{r^{i-1}} = 1.$$

Completeness: if all $S_i = 1$, the product is 1.

Soundness: if $S_j \neq 1$, for some $1 \leq j \leq n$, write each $S_i = g_T^{\delta_i}$ for some uniquely determined $\delta_i \in \mathbb{F}_p$. (Note that $\delta_j \neq 0$.) Then

$$\prod_{i=1}^n S_i^{r^{i-1}} = 1 \iff \sum_{i=1}^n \delta_i r^{i-1} = 0.$$

Setting $f(X) := \sum_{i=1}^n \delta_i X^{i-1}$, the RHS becomes $f(r) = 0$. Since $\delta_j \neq 0$, f is a nonzero polynomial of degree $\leq n-1$. Thus $\Pr[f(r) = 0] \leq (n-1)/p$, negligible for cryptographic p and polynomial n (and still negligible under polynomial grinding in the ROM). *Lydia:* More detail is required here, we can't assume that the polynomial f is random, an adversary has control here of $\text{acc}_i, \dots, \text{acc}_n$, which means they can control the polynomial. I'm also not familiar with the term polynomial grinding $\square$

3 A PCD-style accumulation scheme induced by $(\text{Combine}, D_{\mathcal{B}})$

PCD proofs use an accumulation scheme $\text{AS} = (\text{G}_{\text{AS}}, \text{I}_{\text{AS}}, \text{P}_{\text{AS}}, \text{Verify}_{\text{AS}}, D_{\text{AS}})$ in the sense of [BCMS, §4.1]. *Lydia:* Could we add the model from BCMS into the preliminaries, I think this is a key model so will make it easier on the reader and make the document more complete. In our setting, the objects we wish to “accumulate” are exactly the *decomposed accumulators* output by the verifier gadget/**prepare** procedure *Lydia:* I don't think prepare has been introduced yet, maybe don't refer to gadgets here as that is an implementation detail: each such accumulator represents a pending pairing equation of the form

$$e(\text{Eval}_{\mathcal{B}}(\text{lhs}), [\tau]_2) = e(\text{Eval}_{\mathcal{B}}(\text{rhs}), [1]_2).$$

Accumulation corresponds to repeatedly *folding* multiple such pairing checks into a single one using the deterministic combining map **Combine** (Definition 2.5), whose correctness is captured by Lemma 2.1. Since **Combine** is deterministic (given the random oracle and transcript encoding), the “accumulation verifier proof” π_V required by [BCMS, §4.1] is trivial in our instantiation: the accumulation verifier simply recomputes **Combine** and checks equality. This does *not* contradict “succinctness” in the [BCMS] sense: succinctness concerns maintaining a compact accumulator representation and delegating expensive work to the decider (e.g., a single pairing check), not requiring the AS-verifier to do asymptotically less work than the AS-prover. For fixed recursion

depth d , the global union $\mathcal{B} = \bigcup_{\ell \leq d} \mathcal{B}_\ell$ is fixed at setup time and therefore contributes only a constant-sized fixed-base component to the accumulator (but the global union $\mathcal{B}$ does grow linearly in the number of distinct aggregated relations).

Items and predicate. An AS-*item* is a decomposed accumulator $q \in \text{DA}_{\mathcal{B}}$. Define the item predicate induced by our decider as

$$\Phi_{\mathcal{D}}(q) = 1 \iff \text{D}_{\mathcal{B}}(q) = 1.$$

Intuitively, $\Phi_{\mathcal{D}}(q) = 1$ means that the pairing equation encoded by q is satisfied.

Definition 3.1 (Concrete AS for accumulator folding). *Fix the random oracle H and transcript encoding PI used in Definition 2.5 Lydia:also mention $\mathcal{B}$. Lydia: AS should be introduced generically, instead of as a concrete instantiation, then the generic version used here. If this isn't possible I would just remove AS as a building block and put the instantiation straight here Define AS as follows (we specialize to the binary-folding arity used by our recursion, i.e., two items and two old accumulators per step):*

- $\text{G}_{\text{AS}}(1^\lambda)$ outputs public parameters Lydia: τ is specific to an instantiation of AS

$$\mathbb{P}_{\text{AS}} := (H, \text{PI}, \mathcal{B}, [\tau]_2, [1]_2),$$

i.e., all public data needed to run **Combine** and $\text{D}_{\mathcal{B}}$.

- $\text{I}_{\text{AS}}(\mathbb{P}_{\text{AS}}, \mathbb{P}_{\Phi}, i_{\Phi})$ outputs keys $(\text{apk}, \text{avk}, \text{dk})$ that contain no secrets and can be taken to be pointers into $\mathbb{P}_{\text{AS}}$ (equivalently, empty strings). We keep the notation to match [BCMS].

- $\text{P}_{\text{AS}}(\text{apk}, [q_1, q_2], [a_1, a_2])$ deterministically computes the folded accumulator

$$\text{acc} := \text{Combine}(q_1, a_1, q_2, a_2),$$

and outputs the trivial verifier proof $\pi_V := \perp$. For the base case $\perp$ (corresponding to $n = m = 0$ in [BCMS, §4.1]), it outputs $\text{acc} := \text{ZeroAcc}$ and $\pi_V := \perp$.

- $\text{Verify}_{\text{AS}}(\text{avk}, [q_1, q_2], [a_1, a_2], \text{acc}, \pi_V) = 1$ iff acc is the correct deterministic recomputation target, i.e.,

$$\text{acc} = \text{Combine}(q_1, a_1, q_2, a_2),$$

(and for the base case $\perp$, iff $\text{acc} = \text{ZeroAcc}$). In particular, π_V carries no information and is ignored beyond distinguishing the base case.

- $\text{D}_{\text{AS}}(\text{dk}, \text{acc}) := \text{D}_{\mathcal{B}}(\text{acc})$, i.e., accept iff the pairing equation encoded by acc holds.

Lydia:the definitions from BCMS should be included

Lemma 3.1 (AS completeness and soundness). *The scheme AS of Definition 3.1 is:*

1. perfectly complete for $\Phi_{\mathcal{D}}$;
2. sound for $\Phi_{\mathcal{D}}$ with error probability $\text{negl}(\lambda)$.

Proof. Perfect completeness of $\text{Verify}_{\text{AS}}$ is immediate because P_{AS} sets acc to the exact deterministic value prescribed by **Combine**. Moreover, if $\Phi_{\mathcal{D}}(q_1) = \Phi_{\mathcal{D}}(q_2) = \Phi_{\mathcal{D}}(a_1) = \Phi_{\mathcal{D}}(a_2) = 1$, then Lemma 2.1(1) gives $\text{D}_{\text{AS}}(\text{dk}, \text{acc}) = \text{D}_{\mathcal{B}}(\text{Combine}(\dots)) = 1$.

For soundness, suppose $\text{Verify}_{\text{AS}} = 1$. Then necessarily $\text{acc} = \text{Combine}(q_1, a_1, q_2, a_2)$. If, in addition, $\text{D}_{\text{AS}}(\text{dk}, \text{acc}) = 1$, then $\text{D}_{\mathcal{B}}(\text{Combine}(q_1, a_1, q_2, a_2)) = 1$, and Lemma 2.1(2) implies $\text{D}_{\mathcal{B}}(q_1) = \text{D}_{\mathcal{B}}(q_2) = \text{D}_{\mathcal{B}}(a_1) = \text{D}_{\mathcal{B}}(a_2) = 1$ except with probability $\text{negl}(\lambda)$, i.e., $\Phi_{\mathcal{D}}$ holds for all accumulated inputs. $\square$

4 Underlying proof system and proof-accumulators

For each level $\ell \geq 0$, let ARG_ℓ *Lydia:silly question sorry but why does the proof system change at each level or is this just a generalisation* be the proof system used to prove satisfiability of a relation R_ℓ . *Lydia:for completeness we should probably have a definition for a proof system in the preliminaries or in an appendix* Let $\text{Verify}_\ell(x, \pi) \in \{0, 1\}$ denote its verification predicate. Let X_ℓ denote the statement space, Ω_ℓ the witness space and Π_ℓ the proof space for level ℓ , i.e.,

$$R_\ell \subseteq X_\ell \times \Omega_\ell, \quad \text{Verify}_\ell : X_\ell \times \Pi_\ell \rightarrow \{0, 1\}.$$

Proof-accumulator function. *Lydia:reference the decomposed accumulator from above* For each level ℓ , define a deterministic map

$$\text{PA}_\ell : X_\ell \times \Pi_\ell \rightarrow \text{DA}_\mathcal{B} \times \{0, 1\}$$

that, on input a statement x and purported proof π , runs the level- ℓ PLONK verification logic up to (but excluding) the final KZG pairing check. It outputs a pair

$$\text{PA}_\ell(x, \pi) = (\text{acc}, b), \quad \text{acc} = (\text{lhs}, \text{rhs}),$$

where b indicates whether the verifier constraints are satisfied (equivalently, whether a valid proof can be constructed for the relation at level ℓ) *Lydia:is this the partial checks or the full checks?*.

If $b = 1$, then acc is the decomposed accumulator corresponding to the terminal verifier equation, and verifier acceptance is characterized by

$$\text{Verify}_\ell(x, \pi) = 1 \iff \text{D}_\mathcal{B}(\text{acc}) = 1.$$

If $b = 0$, then the input (x, π) violates the verifier constraints and no valid accumulator (and hence no valid proof) exists.

Equivalently, when $b = 1$, the accumulator acc is obtained by rewriting the final KZG verification equation of the PLONK prover into the canonical pairing form *Lydia:this is a concrete instantiation detail, I think the rewriting the definition of the decomposed accumulator should mean its not necessary anymore though*,

$$e(\text{Eval}_\mathcal{B}(\text{lhs}), [\tau]_2) = e(\text{Eval}_\mathcal{B}(\text{rhs}), [1]_2),$$

as produced by the verifier gadget / **prepare** procedure, which explicitly collects variable-base terms and aggregates fixed-base contributions into the scalar map over $\mathcal{B}$.

Lydia:instead of being assumptions these should be part of the definition of the proof system used

Assumption 4.1 (Perfect completeness of ARG_ℓ). *For each $\ell \geq 0$, ARG_ℓ has perfect completeness for the relations defined below in section 5.*

Assumption 4.2 (Knowledge soundness of ARG_ℓ). *For each $\ell \geq 0$, ARG_ℓ is a knowledge argument for the relations defined below in section 5.*

PA $_\ell$ as a relation. Define the relation $R_\ell^{\text{PA}} \subseteq (X_\ell \times \Pi_\ell) \times (\Omega_\ell \times \text{DA}_\mathcal{B})$ such that $R_\ell^{\text{PA}}(x, \pi; \omega, \rho) = 1$ iff ω satisfies the gadget constraints and the gadget outputs ρ . *Lydia:above avoid mentioning gadgets as they are an implementation detail, also its not clear exactly what the gadget constraints are*

Remark 4.1 (Determinism / consistency of PA $_\ell$). *For all (x, π) , if $\text{PA}_\ell(x, \pi) = (\rho, 1)$ then there exists ω such that $R_\ell^{\text{PA}}(x, \pi; \omega, \rho) = 1$. If $R_\ell^{\text{PA}}(x, \pi; \omega, \rho) = 1$ for some ω , then $\text{PA}_\ell(x, \pi) = (\rho, 1)$ (same ρ) by determinism.*

Lemma 4.1 (Verifier–accumulator consistency: forward direction). *For any $\ell \geq 0$ and any (x, π) , let $(\rho, b) := \text{PA}_\ell(x, \pi)$. If $\text{Verify}_\ell(x, \pi) = 1$, then $b = 1$ and $\text{D}_\mathcal{B}(\rho) = 1$.*

Proof. If $\text{Verify}_\ell(x, \pi) = 1$ then non-pairing checks must pass, hence $b = 1$. When $b = 1$, acceptance is equivalent to $\text{D}_\mathcal{B}(\rho) = 1$ by the definition of PA $_\ell$. $\square$

Lemma 4.2 (Verifier–accumulator consistency: reverse direction). *For any $\ell \geq 0$ and any (x, π) , if there exist (ω, ρ) such that $R_\ell^{\text{PA}}(x, \pi; \omega, \rho) = 1$ and $\text{D}_\mathcal{B}(\rho) = 1$, then $\text{Verify}_\ell(x, \pi) = 1$.*

Proof. By Remark 4.1, $R_\ell^{\text{PA}}(x, \pi; \omega, \rho) = 1$ implies $\text{PA}_\ell(x, \pi) = (\rho, 1)$. When the bit is 1, verifier acceptance is equivalent to $\text{D}_\mathcal{B}(\rho) = 1$, which holds by premise. $\square$

5 PCD-style recursion: relations, predicate, and verifier

5.1 Leaf relation (layer $\ell = 0$)

Fix $t, k \in \mathbb{N}$ and an underlying relation $\mathcal{R} \subseteq \mathbb{F}^k \times \mathbb{F}^t$, where $x \in \mathbb{F}^k$ is the (would-be) public input and $w \in \mathbb{F}^t$ is the witness. We define a derived relation R_0 whose *public instance* is a single field element $s \in \mathbb{F}$ that serves as a digest of x .

For $t, k \in \mathbb{N}$, define:

$$x_0 = (s) \in \mathbb{F}, \quad w_0 = (x, w) \in \mathbb{F}^k \times \mathbb{F}^t,$$

and

$$R_0(x_0, w_0) = 1 \iff (s = \text{Hash}(\text{encode}(x))) \wedge \mathcal{R}(x, w) = 1.$$

Remark 5.1. *Intuitively, s is a commitment/digest to the vector x . The relation R_0 asserts that there exist (x, w) such that $\mathcal{R}(x, w) = 1$ and the public digest s is consistent with x via $s = \text{Hash}(\text{encode}(x))$.*

5.2 Aggregation relations for $\ell \geq 1$

For $\ell \geq 0$, define a layer-dependent statement constructor

$$\text{Stmt}_\ell(s, \text{acc}) := \begin{cases} (s) & \text{if } \ell = 0, \\ (s, \text{acc}, \ell) & \text{if } \ell \geq 1. \end{cases}$$

For $\ell \geq 1$, the public instance is

$$x_\ell := \text{Stmt}_\ell(s, \text{acc}) = (s, \text{acc}, \ell),$$

where $s \in \mathbb{F}$ is the node state, $\text{acc} \in \text{DA}_\mathcal{B}$ is the carried accumulator (encoded via $\text{PI}(\text{acc})$ in the public inputs), and ℓ is a level tag.

Witness. For $\ell \geq 1$, we split the witness into (i) the *local data* used by the compliance predicate φ , and (ii) the (possibly trivial) proof π_V for the accumulator step:

$$w_\ell := (z_{\text{loc}}, \pi_V).$$

The local data z_{loc} comprises left/right child information and the data needed to extract accumulation *items* from child proofs:

$$z_{\text{loc}} := (s_L, s_R, \pi_L, \pi_R, \text{acc}_L, \text{acc}_R, \omega_L, \omega_R, \rho_L, \rho_R),$$

where:

- $s_L, s_R \in \mathbb{F}$ are the child states;
- π_L, π_R are proofs for the child instances at layer $\ell - 1$;
- $\text{acc}_L, \text{acc}_R \in \text{DA}_{\mathcal{B}}$ are the child *carried* accumulators;
- ρ_L, ρ_R are the *items* output by verifying the child proofs (via the $R_{\ell-1}^{\text{PA}}$ relations);
- ω_L, ω_R are auxiliary witnesses for those $R_{\ell-1}^{\text{PA}}$ checks.

When $\ell = 1$, there are no carried child accumulators; we neutralize by setting $\text{acc}_L = \text{acc}_R := \text{ZeroAcc}$. (The value π_V will be used only in the accumulator step below; in our concrete AS it is trivial.)

Neutralization. Define

$$\text{acc}'_L := \begin{cases} \text{ZeroAcc} & \text{if } \ell = 1, \\ \text{acc}_L & \text{if } \ell > 1, \end{cases} \quad \text{acc}'_R := \begin{cases} \text{ZeroAcc} & \text{if } \ell = 1, \\ \text{acc}_R & \text{if } \ell > 1. \end{cases}$$

Define the child instances (using the neutralized carried accumulators):

$$x_L := \text{Stmt}_{\ell-1}(s_L, \text{acc}'_L), \quad x_R := \text{Stmt}_{\ell-1}(s_R, \text{acc}'_R).$$

Note that we can just not witness any carried accumulators at all for the base relations ($\ell = 1$).

Definition 5.1 (Local compliance predicate φ). *Let $\mathcal{F}$ contain the single predicate φ defined as follows. On input an outgoing message $z := (s, \ell)$, local data z_{loc} , and two incoming messages $z_1 := (s_L, \ell - 1)$, $z_2 := (s_R, \ell - 1)$ (or both $\perp$ when $\ell = 0$), define:*

- *If $\ell = 0$: parse $z_{\text{loc}} = (x, w)$ and accept iff $z_1 = z_2 = \perp$ and $R_0((s), (x, w)) = 1$. *Lydia:R₀ is derived generically from R, do we need to specify here that it has been derived from R_0^{PA}**
- *If $\ell \geq 1$: parse*

$$z_{\text{loc}} = (s_L, s_R, \pi_L, \pi_R, \text{acc}_L, \text{acc}_R, \omega_L, \omega_R, \rho_L, \rho_R)$$

and accept iff:

1. $s = \text{Hash}(s_L, s_R)$;
2. $R_{\ell-1}^{\text{PA}}(x_L, \pi_L; \omega_L, \rho_L) = 1$ and $R_{\ell-1}^{\text{PA}}(x_R, \pi_R; \omega_R, \rho_R) = 1$;
3. *the incoming messages match the parsed children: $z_1 = (s_L, \ell - 1)$ and $z_2 = (s_R, \ell - 1)$.*

(Here x_L, x_R use the neutralized carried accumulators $\text{acc}'_L, \text{acc}'_R$ defined above.)

Definition 5.2 (Recursive relations R_ℓ). For each $\ell \in \{0, \dots, d\}$, define $R_\ell \subseteq \mathbf{X}_\ell \times \Omega_\ell$ as follows.

- **Layer 0.** The statement is $x_0 = (s)$ and the witness is $w_0 = (x, w)$. Define $R_0((s), (x, w)) = 1$ as in Section 5.
- **Layer $\ell \geq 1$.** The statement is $x_\ell = (s, \text{acc}, \ell)$. The witness is $w_\ell = (z_{\text{loc}}, \pi_V)$, where z_{loc} is parsed as in Section 5 (including child proofs, carried child accumulators, and ρ_L, ρ_R). Let $z := (s, \ell)$, $z_1 := (s_L, \ell - 1)$, $z_2 := (s_R, \ell - 1)$ be the outgoing/incoming messages implied by that parse (and the neutralization rule for $\ell = 1$). Then $R_\ell(x_\ell, w_\ell) = 1$ iff both:
 1. $\varphi(z, z_{\text{loc}}, z_1, z_2) = 1$; and
 2. $\text{Verify}_{\text{AS}}(\text{avk}, [\rho_L, \rho_R], [\text{acc}'_L, \text{acc}'_R], \text{acc}, \pi_V) = 1$, where $\text{acc}'_L, \text{acc}'_R$ are the neutralized carried accumulators (Definition in Section 5).

Recursive circuit indices. For each $\ell \in \{0, \dots, d\}$, let $i^{(\ell)}$ denote the ARG-index for the circuit corresponding to the PCD recursive check at depth ℓ . Concretely, $i^{(\ell)}$ is the circuit that:

1. checks $\varphi(z, z_{\text{loc}}, z_1, z_2) = 1$ (with $z = (s, \ell)$);
2. if $\ell \geq 1$, checks the AS verification step combining two *items* and two *old accumulators*:

$$\text{Verify}_{\text{AS}}(\text{avk}, [\rho_L, \rho_R], [\text{acc}'_L, \text{acc}'_R], \text{acc}, \pi_V) = 1.$$

This circuit is the formalization of the intended **AggCircuit** behavior under PCD-style decomposition: “local checks” (φ) plus “accumulator step” (AS-verification).

Terminal verifier. The *terminal verifier* (run only once at the end of recursion) runs in addition to the SNARK verifier the decider on the carried accumulator contained in the terminal statement:

$$\text{Verify}_\ell^{\text{term}}(x_\ell, \pi) = 1 \iff \text{Verify}_\ell(x_\ell, \pi) = 1 \wedge D_{\mathcal{B}}(\text{acc}) = 1,$$

where $x_\ell = (s, \text{acc}, \ell)$ for $\ell \geq 1$. This preserves the “deferred pairing check”: pairing work is delegated to the terminal decider rather than performed inside each recursive step.

Remark 5.2 (Root fixed-base MSM). Computing the fixed-base MSM inside the root circuit instead of deferring it does not change the value of $D_{\mathcal{B}}$ (Lemma 2.1), and therefore does not affect the completeness/soundness proofs; it only relocates the evaluation of $\text{Eval}_{\mathcal{B}}$ from the recursive verifier into the terminal (root) aggregated proof that the recursive verifier checks.

6 Completeness

Theorem 6.1 (PCD-style completeness from ARG perfect completeness and AS completeness).

Assume:

- Assumption 4.1 (perfect completeness of ARG_ℓ for all $\ell \leq d$);
- Lemma 3.1(1) (AS completeness for $\Phi_{\mathcal{D}}$).

Then the recursive verifier is perfectly complete at every layer $\ell \leq d$ under the standard PCD completeness precondition.

Proof. Fix a layer ℓ . Consider any adversary A outputting data for an honest prover call at layer ℓ :

$$(\varphi, z, z_{\text{loc}}, (z_1, (\pi_1, \text{acc}_1)), (z_2, (\pi_2, \text{acc}_2))),$$

where $z = (s, \ell)$, $z_i = (s_i, \ell - 1)$ (or $\perp$ in the base case), and (π, acc) is produced honestly by the prover.

Define the PCD completeness bad event (for this ℓ):

$$\text{Bad}_{\text{PCD-compl}}^{(\ell)} := \left[\varphi \in \mathcal{F} \wedge (z_1 = z_2 = \perp \vee \forall i \in \{1, 2\}, \text{Verify}_{\ell-1}(x_i, \pi_i) = 1 \wedge D_{\mathcal{B}}(\text{acc}_i) = 1) \right. \\ \left. \wedge \varphi(z, z_{\text{loc}}, z_1, z_2) = 1 \wedge (\text{Verify}_{\ell}(x_{\ell}, \pi) = 0 \vee D_{\mathcal{B}}(\text{acc}) = 0) \right],$$

where x_i is the child instance (using acc'_i ; at $\ell = 1$ these are **ZeroAcc**) and $x_{\ell} = (s, \text{acc}, \ell)$.

We prove $\mathbb{P}[\text{Bad}_{\text{PCD-compl}}^{(\ell)}] = 0$ by reduction to AS completeness.

Step 1: Assume toward contradiction a PCD completeness breaker. Assume there exists an adversary A such that the PCD *completeness failure event* (i.e., $\text{Bad}_{\text{PCD-compl}}^{(\ell)}$) occurs with probability $p > 0$, namely

$$p := \mathbb{P}[\text{Bad}_{\text{PCD-compl}}^{(\ell)}] > 0.$$

We will build an adversary B for the AS-completeness experiment such that

$$\text{Bad}_{\text{PCD-compl}}^{(\ell)} \subseteq \text{Bad}_{\text{AS-compl}}^{(B)}.$$

Consequently,

$$\mathbb{P}[\text{Bad}_{\text{PCD-compl}}^{(\ell)}] \leq \mathbb{P}[\text{Bad}_{\text{AS-compl}}^{(B)}].$$

By Lemma 3.1(1) (perfect completeness of AS), the right-hand side is 0 for every B , and hence $\mathbb{P}[\text{Bad}_{\text{PCD-compl}}^{(\ell)}] = 0$, contradicting $p > 0$.

Step 2: Define the AS adversary B . On input $(\mathbb{P}, \mathbb{P}_{\text{AS}})$, where $\mathbb{P}$ denotes the public parameters of the PCD experiment (in particular, it includes whatever public parameters/indices are needed to interpret and verify ARG_{ℓ} and $\text{ARG}_{\ell-1}$ proofs), B runs A on the combined public input

$$\mathbb{P}^* := (\mathbb{P}, \mathbb{P}_{\text{AS}})$$

to obtain

$$(\varphi, z, z_{\text{loc}}, (z_1, (\pi_1, \text{acc}_1)), (z_2, (\pi_2, \text{acc}_2))) \leftarrow A(\mathbb{P}^*).$$

For $\ell \geq 1$, B parses z_{loc} to recover the proof-accumulators ρ_1, ρ_2 (as the corresponding components of z_{loc} , i.e., the values purported to be output by the $\text{PA}_{\ell-1}$ gadgets for the two child proofs), and outputs the AS instance:

$$\text{items } [q_1, q_2] = [\rho_1, \rho_2], \quad \text{old accumulators } [a_1, a_2] = [\text{acc}'_1, \text{acc}'_2].$$

(At $\ell = 1$, $\text{acc}'_1 = \text{acc}'_2 = \text{ZeroAcc}$ by definition.)

Step 3: Expand “PCD verifier rejects” into two failure modes. By definition, $\text{Verify}_{\ell}(x_{\ell}, \pi) = 1$ and $D_{\mathcal{B}}(\text{acc}) = 1$ are the two conjuncts. So if the PCD verifier rejects, then either $\text{Verify}_{\ell}(x_{\ell}, \pi) = 0$ or $D_{\mathcal{B}}(\text{acc}) = 0$.

Step 4: Handle the base case. If $\ell = 0$, the claim is immediate from Assumption 4.1 and the fact that the carried accumulator is ZeroAcc , for which $D_B(\text{ZeroAcc}) = 1$.

Now assume $\ell \geq 1$. By Definition 5.1, in the $\ell \geq 1$ branch the incoming messages must parse as $z_1 = (s_L, \ell-1)$ and $z_2 = (s_R, \ell-1)$, so in particular $z_1 \neq \perp$ and $z_2 \neq \perp$ whenever $\varphi(z, z_{\text{loc}}, z_1, z_2) = 1$. Therefore, any execution contributing to $\text{Bad}_{\text{PCD-compl}}^{(\ell)}$ with $\ell \geq 1$ is necessarily in the recursive case. (When $\ell = 1$, the neutralization $\text{acc}'_1 = \text{acc}'_2 = \text{ZeroAcc}$ is enforced by the witness parsing and the definition of x_L, x_R .)

Step 5: From the PCD precondition, deduce AS completeness premise. In the recursive case, the bad-event precondition includes that both child verifiers accept: $\text{Verify}_{\ell-1}(x_i, \pi_i) = 1$ and $D_B(\text{acc}_i) = 1$ for $i \in \{1, 2\}$.

Since $\varphi(z, z_{\text{loc}}, z_1, z_2) = 1$, Definition 5.1 implies that for each child i there exists ω_i such that $R_{\ell-1}^{\text{PA}}(x_i, \pi_i; \omega_i, \rho_i) = 1$. By Remark 4.1 (determinism/consistency of PA), this forces $\text{PA}_{\ell-1}(x_i, \pi_i) = (\rho_i, 1)$.

Now apply Lemma 4.1 to $\text{Verify}_{\ell-1}(x_i, \pi_i) = 1$ to obtain $D_B(\rho_i) = 1$, i.e., $\Phi_D(q_i) = 1$ for the AS items $q_i := \rho_i$.

Also $D_B(\text{acc}'_i) = 1$ holds for the old accumulators $a_i := \text{acc}'_i$ (either because $\text{acc}'_i = \text{acc}_i$ and $D_B(\text{acc}_i) = 1$, or because $\text{acc}'_i = \text{ZeroAcc}$). Thus the AS completeness premise holds for B 's instance.

Step 6: The honest ARG_ℓ prover must verify under the completeness precondition.

In the experiment defining PCD completeness, the proof π at layer ℓ is generated by the *honest* prover for ARG_ℓ . Under the bad-event precondition we have $\varphi(z, z_{\text{loc}}, z_1, z_2) = 1$, and the witness values in z_{loc} certify the two $\text{PA}_{\ell-1}$ gadget relations and (in the circuit) the deterministic $\text{Verify}_{\text{AS}}$ recomputation check. Therefore, the layer- ℓ circuit relation $R_\ell(x_\ell, w_\ell) = 1$ is satisfiable for the witness w_ℓ used by the honest prover.

Moreover, in the honest recursive prover, the pair (acc, π_V) appearing in the statement/witness is computed as $(\text{acc}, \pi_V) \leftarrow \text{P}_{\text{AS}}(\text{apk}, [\rho_1, \rho_2], [\text{acc}'_1, \text{acc}'_2])$ (or $(\text{ZeroAcc}, \perp)$ in the base case), hence the AS-verification conjunct in Definition 5.2 is satisfied by construction.

By perfect completeness of ARG_ℓ (Assumption 4.1), it follows that $\text{Verify}_\ell(x_\ell, \pi) = 1$ always.

Moreover, since the circuit $i^{(\ell)}$ explicitly enforces the AS-verification step, whenever $\text{Verify}_\ell(x_\ell, \pi) = 1$ we necessarily also have

$$\text{Verify}_{\text{AS}}(\text{avk}, [\rho_1, \rho_2], [\text{acc}'_1, \text{acc}'_2], \text{acc}, \pi_V) = 1.$$

Therefore, in any execution contributing to $\text{Bad}_{\text{PCD-compl}}^{(\ell)}$, the only possible reason for rejection is the remaining conjunct:

$$D_B(\text{acc}) = 0.$$

Step 7 (bridge to the AS completeness experiment). Now consider the AS-completeness experiment with adversary B . By construction, the pair $(\mathbb{P}, \mathbb{P}_{\text{AS}})$ given to B in the AS experiment is sampled identically to the pair $(\mathbb{P}, \mathbb{P}_{\text{AS}})$ used in the PCD completeness experiment (and B invokes A on $\mathbb{P}^* = (\mathbb{P}, \mathbb{P}_{\text{AS}})$ in both cases). Therefore, the joint distribution of A 's output $(\varphi, z, z_{\text{loc}}, (z_1, (\pi_1, \text{acc}_1)), (z_2, (\pi_2, \text{acc}_2)))$ is identical in the AS experiment and in the PCD experiment. In particular, whenever $\text{Bad}_{\text{PCD-compl}}^{(\ell)}$ occurs, the derived AS instance output by B satisfies the AS completeness premise:

$$\Phi_D(q_1) = \Phi_D(q_2) = \Phi_D(a_1) = \Phi_D(a_2) = 1,$$

where $[q_1, q_2] = [\rho_1, \rho_2]$ and $[a_1, a_2] = [\text{acc}'_1, \text{acc}'_2]$ are as in Step 2.

Moreover, by Step 6, on $\text{Bad}_{\text{PCD-compl}}^{(\ell)}$ we have $D_B(\text{acc}) = 0$, and since $D_{\text{AS}}(\text{dk}, \text{acc}) = D_B(\text{acc})$, it follows that $D_{\text{AS}}(\text{dk}, \text{acc}) = 0$. Therefore, on every execution where $\text{Bad}_{\text{PCD-compl}}^{(\ell)}$ occurs, B produces an AS instance for which the AS completeness premise holds but the conclusion fails.

Equivalently, letting

$$\text{Bad}_{\text{AS-compl}}^{(B)} := \left[\Phi_D(q_1) = \Phi_D(q_2) = \Phi_D(a_1) = \Phi_D(a_2) = 1 \wedge D_{\text{AS}}(\text{dk}, \text{acc}) = 0 \right],$$

we have the containment

$$\text{Bad}_{\text{PCD-compl}}^{(\ell)} \subseteq \text{Bad}_{\text{AS-compl}}^{(B)},$$

and hence

$$\mathbb{P}[\text{Bad}_{\text{PCD-compl}}^{(\ell)}] \leq \mathbb{P}[\text{Bad}_{\text{AS-compl}}^{(B)}].$$

By Lemma 3.1(1), $\mathbb{P}[\text{Bad}_{\text{AS-compl}}^{(B)}] = 0$, so $\mathbb{P}[\text{Bad}_{\text{PCD-compl}}^{(\ell)}] = 0$. □

7 Knowledge soundness

Theorem 7.1 (PCD-style knowledge soundness from ARG knowledge soundness and AS soundness).

Assume:

- *Assumption 4.2* (ARG_ℓ knowledge soundness with vectors+aux+stat. closeness for all $\ell \leq d$);
- *Lemma 3.1(2)* (AS soundness for Φ_D).

Then the recursive aggregation scheme is knowledge sound for constant depth d :

1. *whenever the root verifier accepts, the extractor outputs a full φ -compliant binary transcript/tree (down to leaf layer 0, i.e., along d recursion steps) except with probability $\text{negl}(\lambda)$;*
2. *the output distribution on the top-level claim is statistically close to the malicious prover's output.*

Proof. Fix a malicious prover $\tilde{P}$ that outputs a purported $\ell = d$ proof:

$$(\varphi, z_{\text{root}} = (s_{\text{root}}, d), (\pi_{\text{root}}, \text{acc}_{\text{root}})).$$

Let μ denote the acceptance probability of the recursive verifier at the root:

$$\mu := \mathbb{P}[\text{Verify}_d(x_d, \pi_{\text{root}}) = 1 \wedge D_B(\text{acc}_{\text{root}}) = 1].$$

Tree normalization. We build a binary tree T where each node u carries:

$$(\ell(u), s(u), z_{\text{loc}}(u), \pi(u), \text{acc}(u)),$$

with $\ell(\text{root}) = d$ and children always at layer one less.

Extractor stages $E_0, E_1, \dots, E_{d+1}$

Let $\ell_T(j)$ be the set of nodes at depth j (root at depth 1).

Base extractor E_0 . On input system parameters, run $\tilde{P}$ and output the single-node tree T_0 containing the root label $(d, s_{\text{root}}, \perp, \pi_{\text{root}}, \mathbf{acc}_{\text{root}})$.

Derived prover $\tilde{P}_j$. Assume E_{j-1} is defined. Let $\ell_j := d - (j - 1)$ be the layer at depth j . Define $\tilde{P}_j$ to output a vector of ARG_{ℓ_j} statements consisting of the nodes $v \in \ell_{T_{j-1}}(j)$:

- instances $x(v) = (s(v), \mathbf{acc}(v), \ell_j)$;
- proofs $\pi(v)$;
- auxiliary output $\mathbf{aux} := (\varphi, T_{j-1})$.

Define E_j using the ARG_{ℓ_j} extractor. Apply the ARG_{ℓ_j} knowledge extractor (Assumption 4.2) to $\tilde{P}_j$ to extract witnesses for all depth- j nodes. Parse each extracted witness into

$$w(v) = (s_L, s_R, \pi_L, \pi_R, \mathbf{acc}_L, \mathbf{acc}_R, \omega_L, \omega_R, \rho_L, \rho_R, \pi_V)$$

(and in the base case $\ell_j = 0$, parse $z_{\text{loc}} = (x, w)$).

Then set the local data of v to the corresponding extracted value:

$$z_{\text{loc}}(v) := \begin{cases} (x, w) & \text{if } \ell_j = 0, \\ (s_L, s_R, \pi_L, \pi_R, \mathbf{acc}_L, \mathbf{acc}_R, \omega_L, \omega_R, \rho_L, \rho_R) & \text{if } \ell_j \geq 1, \end{cases}$$

and extend the tree by attaching two children at layer $\ell_j - 1$ with labels

$(\ell_j - 1, s_L, \perp, \pi_L, \mathbf{acc}'_L)$ and $(\ell_j - 1, s_R, \perp, \pi_R, \mathbf{acc}'_R)$ (omitting children when $\ell_j = 0$).

Final extractor. Let $E := E_{d+1}$. Output $(\varphi, (\pi_{\text{root}}, \mathbf{acc}_{\text{root}}), T_{d+1})$.

Correctness/compliance

We prove by induction on $j \in \{0, \dots, d + 1\}$ that with probability at least $\mu - \text{negl}(\lambda)$:

T_j is φ -compliant up to depth j , and every node v in T_j satisfies $\text{Verify}_{\ell(v)}(x(v), \pi(v)) = 1$ and $D_{\mathcal{B}}(\mathbf{acc}(v)) = 1$.

Since d is a fixed constant, the total number of extractor invocations and AS-soundness applications over the whole tree is $O(2^d)$, i.e., constant. Thus we may union bound over all nodes/levels and still absorb the resulting constant factor into $\text{negl}(\lambda)$.

Base case ($j = 0$). If the root verifier accepts with probability μ , then $\text{Verify}_d(x_d, \pi_{\text{root}}) = 1$ and $D_{\mathcal{B}}(\mathbf{acc}_{\text{root}}) = 1$ with probability μ . Compliance up to depth 0 is vacuous.

Inductive step. Assume the claim holds for $j - 1$, prove for j . Let $\ell_j = d - (j - 1)$.

Step A (extract witnesses for depth- j nodes). By induction hypothesis, with probability $\mu - \text{negl}(\lambda)$, every $v \in \ell_{T_{j-1}}(j)$ has an accepted ARG_{ℓ_j} proof. Thus Assumption 4.2 yields extracted witnesses $w(v)$ such that the recursive circuit at layer ℓ_j is satisfied.

Step B (from circuit satisfaction to φ -compliance and AS-step acceptance). Circuit satisfaction implies:

1. $\varphi(z(v), z_{\text{loc}}(v), z_1, z_2) = 1$ (so local compliance holds once children are attached);

2. if $\ell_j \geq 1$, then $\text{Verify}_{\text{AS}}(\text{avk}, [\rho_L, \rho_R], [\text{acc}'_L, \text{acc}'_R], \text{acc}(v), \pi_V) = 1$ for the parsed witness values.

Thus E_j 's extension makes every depth- j node φ -compliant.

Step C (use AS soundness to validate children and enable the next extraction). For any recursive node v (so $\ell_j \geq 1$):

- by induction hypothesis, $D_{\mathcal{B}}(\text{acc}(v)) = 1$;
- by Step B, $\text{Verify}_{\text{AS}}(\dots) = 1$.

Apply AS soundness (Lemma 3.1(2)) to each such node v ; by the constant-depth union bound noted above, except with overall probability $\text{negl}(\lambda)$ we have $D_{\mathcal{B}}(\rho_L) = D_{\mathcal{B}}(\rho_R) = D_{\mathcal{B}}(\text{acc}'_L) = D_{\mathcal{B}}(\text{acc}'_R) = 1$ for all nodes at depth j .

Now combine:

- from φ (Definition 5.1) we have $R_{\ell_j-1}^{\text{PA}}(x_L, \pi_L; \omega_L, \rho_L) = 1$ (and similarly on the right);
- from AS soundness we have $D_{\mathcal{B}}(\rho_L) = 1$.

By Lemma 4.2, $\text{Verify}_{\ell_j-1}(x_L, \pi_L) = 1$ and $\text{Verify}_{\ell_j-1}(x_R, \pi_R) = 1$. Together with $D_{\mathcal{B}}(\text{acc}'_L) = D_{\mathcal{B}}(\text{acc}'_R) = 1$, this shows each attached child would be accepted by the recursive verifier at layer $\ell_j - 1$, enabling the next level of ARG extraction.

Step D. Thus compliance up to depth j and the invariants $\text{Verify}_{\ell(\cdot)} = 1$ and $D_{\mathcal{B}} = 1$ hold for all nodes in T_j .

Finish. After $d + 1$ steps we obtain T_{d+1} , whose depth- $(d + 1)$ nodes are at layer 0 and therefore carry extracted witnesses (x, w) satisfying the base relation. Hence the tree is fully φ -compliant down to leaf layer 0.

This proves the first knowledge-soundness requirement.

Statistical closeness

At level 0, E_0 records $\tilde{P}$'s output exactly. At each level $j \geq 1$, we apply an ARG_{ℓ_j} extractor that preserves the marginal distribution on (vector instances, vector proofs, aux) up to statistical distance $\text{negl}(\lambda)$. Since d is constant, composing d such hybrids changes the overall distribution by at most $d \cdot \text{negl}(\lambda) = \text{negl}(\lambda)$. $\square$

8 Remark on “fixed-base MSM deferred to the root”

Deferring fixed-base MSM evaluation into the terminal decider does not affect the above proofs: all recursive steps manipulate *decomposed* accumulators, and the recursive verifier always checks $D_{\mathcal{B}}$ on the carried accumulator. The effect is purely an optimization of where $\text{Eval}_{\mathcal{B}}$ is fully evaluated (only at the terminal decider), not a semantic change.

9 Rollup state transitions

9.1 Context and high-level goal

Shielded rollup model. We implement a *shielded-payment rollup* in the spirit of Zerocash. Users transact over private state where *notes* are represented publicly by *commitments* and spent notes

are represented publicly by *nullifiers*. A client proves in zero-knowledge that it is spending valid prior notes and reveals only: (i) an *anchor* to a past commitment root, (ii) *nullifiers* (to prevent double-spends), (iii) new *output commitments*.

Swaps (two-party atomic exchange). Besides ordinary shielded transfers, the rollup supports a two-party atomic swap primitive. A swap consists of two client bundles (one per counterparty) that must be included together as siblings in the leaf rollup step. Atomicity is enforced publicly by requiring both bundles to carry the same nonzero *swap-terms digest* st . Each leg carries its own expiry vto and must satisfy $blk \leq vto$ for the batch block number blk . When a match holds, both bundles are applied exactly as normal transfers (nullifier insertions and commitment insertions); otherwise the pair is rejected.

What the rollup circuit does (and does not) enforce. The rollup circuit does *not* re-verify private spend logic. Instead it treats each client proof as an opaque object identified by a single public input called the *instance digest* $inst(u)$, and enforces only the deterministic public state transition induced by the bundle.

Public state as set-like sparse Merkle maps. The rollup maintains three insert-only maps:

- **Commitment set C :** membership means “this commitment exists”.
- **Nullifier set N :** membership means “this nullifier is spent”.
- **Historic roots set H :** membership means “this commitment root is an allowed anchor”.

Each map is a sparse Merkle map keyed by field elements, with default value 0 and present value 1.

Recursion summary state. Define the leaf tag by

$$\text{LeafTag}(u) := \text{inst}(u) \in \mathbb{F}.$$

A recursion node summarizes a subtree by

$$\sigma = (c_{\text{pre}}, c_{\text{post}}, n_{\text{pre}}, n_{\text{post}}, s, r_{\text{hist}}, \text{blk}),$$

where $c_{\text{pre/post}}$ and $n_{\text{pre/post}}$ are the before/after roots of C and N , $r_{\text{hist}} = \text{root}(H)$ is the historic-roots root used to validate anchors, blk is the (batch) L2 rollup block number, and s binds the *exact ordered list* of leaf tags. The digest is formed as $s = \text{Hash}(s_L, s_R)$, with base case

$$s = \text{Hash}(\text{LeafTag}(u_L), \text{LeafTag}(u_R)).$$

9.2 Merkle-map interface (MapGadget semantics)

We model a *Merkle map* as a fixed-height sparse binary Merkle tree of height $h = \text{TREE_HEIGHT} = 128$, with leaf values in $\mathbb{F}$. All hashes are Poseidon over $\mathbb{F}$; $\text{Hash}(x, y)$ denotes arity-2 compression.

Key-to-path bits. Given a key $k \in \mathbb{F}$, define $p := \text{Hash}(k, 0) \in \mathbb{F}$. Let $[p] \in \{0, \dots, |\mathbb{F}| - 1\} \subset \mathbb{Z}$ be the canonical integer representative of p . Define the address bits as the low h bits of $[p]$ (little-endian):

$$(b_0, \dots, b_{h-1}) := \text{LowBits}_h([p]) \in \{0, 1\}^h.$$

In-circuit we bit-decompose p to the field bitlength and truncate to h bits (we do *not* require $p < 2^h$).

Default tree. The default leaf value is $d \in \mathbb{F}$ (here $d = 0$). Unset keys are semantically equal to d , and internal default nodes are precomputed by repeated hashing. A witness for $(k \mapsto d)$ is therefore an *exclusion proof*.

Witness verification. A Merkle witness for k is a sibling list $\pi = (\pi_0, \dots, \pi_{h-1}) \in \mathbb{F}^h$. Given (k, v, π) , define $\text{VerifyPath}(k, v, \pi)$ by starting from $x_0 := v$ and iterating:

$$x_{i+1} := \begin{cases} \text{Hash}(x_i, \pi_i) & \text{if } b_i = 0, \\ \text{Hash}(\pi_i, x_i) & \text{if } b_i = 1, \end{cases} \quad \text{VerifyPath}(k, v, \pi) := x_h.$$

The witness is valid w.r.t. root r iff $\text{VerifyPath}(k, v, \pi) = r$.

Lookup and update relations. In-circuit, a map is represented by its current root $\text{repr} \in \mathbb{F}$.

- **Lookup.** Given k and witness (v, π) , constrain

$$\text{VerifyPath}(k, v, \pi) = \text{repr}.$$

- **Update.** To set $k \mapsto v$, reuse one π and constrain

$$\text{VerifyPath}(k, v_{\text{old}}, \pi) = \text{repr} \quad \text{and} \quad \text{VerifyPath}(k, v, \pi) = \text{repr}',$$

then update $\text{repr} \leftarrow \text{repr}'$.

Rollup convention (set-like maps). All maps use $d = 0$. Inserts set values to 1. Freshness is enforced by requiring $\text{get}(M, k) = 0$ before writing $k \mapsto 1$.

9.3 Client bundles and their public effects

Definition 9.1 (Apply (historic-root anchored)). *Let C be the commitment map, N the nullifier map, and H the historic-roots map. We write $\text{Apply}(u, C, N, H) = (C', N')$ iff:*

1. **Anchor is authorized:** $\text{get}(H, \text{rt}(u)) = 1$.

2. **Insert output commitments into C :**

$$C' := \text{ins}(\text{ins}(C, \text{cm}_1(u), 1), \text{cm}_2(u), 1).$$

3. **Insert fresh nullifiers into N (sequential freshness):**

$$\text{get}(N, \text{nf}_1(u)) = 0, \quad N_1 := \text{ins}(N, \text{nf}_1(u), 1), \quad \text{get}(N_1, \text{nf}_2(u)) = 0, \quad N' := \text{ins}(N_1, \text{nf}_2(u), 1).$$

Definition 9.2 (Swap intent and swap-terms digest). *A swap intent is the tuple of all public fields that must agree across the two swap legs, excluding expiry. It binds the identities of both counterparties (A and B) and the full economic terms of the exchange*

Canonical A/B ordering (field elements). *Let $q := |\mathbb{F}|$ and let $[x] \in \{0, \dots, q-1\}$ denote the canonical integer representative of $x \in \mathbb{F}$. Let a swap bundle u carry two public keys $\text{PK}_1(u), \text{PK}_2(u) \in \mathbb{F}$ for the two parties. Define*

$$\text{PK}_A(u) := \begin{cases} \text{PK}_1(u) & \text{if } [\text{PK}_1(u)] \leq [\text{PK}_2(u)], \\ \text{PK}_2(u) & \text{otherwise,} \end{cases} \quad \text{PK}_B(u) := \begin{cases} \text{PK}_2(u) & \text{if } [\text{PK}_1(u)] \leq [\text{PK}_2(u)], \\ \text{PK}_1(u) & \text{otherwise.} \end{cases}$$

In-circuit, this comparison must be implemented using an explicit range-checked integer representation and comparator so both parties compute the same ordering.

Intent fields. Concretely, a swap bundle u specifies:

- token identifiers $\text{token}_A(u), \text{token}_B(u) \in \mathbb{F}$;
- amounts $\text{value}_A(u), \text{value}_B(u) \in \mathbb{F}$;
- a swap nonce $\text{nonce}(u) \in \mathbb{F}$.

Define the canonical intent tuple

$$\text{Intent}(u) := (\text{PK}_A(u), \text{PK}_B(u), \text{token}_A(u), \text{value}_A(u), \text{token}_B(u), \text{value}_B(u), \text{nonce}(u)) \in \mathbb{F}^7.$$

Define the swap-terms digest as the domain-separated hash

$$\text{st}(u) := \text{Hash}(\text{Intent}(u)) \in \mathbb{F}.$$

Rollup vs client enforcement. The rollup circuit treats $\text{st}(u)$ as an opaque commitment and does not re-parse or recompute $\text{Intent}(u)$ from additional fields. Instead, correctness requires the client proof system to enforce that the private spend and outputs implement the intended exchange terms committed in $\text{st}(u)$, and that any public swap-intent fields (if exposed) are consistent with $\text{st}(u)$.

Definition 9.3 (Per-leg swap tag (binds intent and expiry)). Let $\text{st}(u) \in \mathbb{F}$ be the swap-terms digest (Definition 9.2). The rollup does not interpret st , but enforces that sw binds st and vto via:

$$\text{sw}(u) := \text{Hash}(\text{st}(u), \text{vto}(u)).$$

The rollup matches swap legs on st (shared across the pair); sw is a per-leg tag that additionally binds the leg-specific expiry vto .

Reserved zero. We reserve $\text{sw}(u) = 0$ to mean “not a swap leg”. Therefore, for a swap leg the circuit must enforce

$$\text{sw}(u) \neq 0.$$

(If $\text{Hash}(\text{st}, \text{vto}) = 0$ occurs, which is negligible for random-oracle-modeled hashes, the client must pick a fresh nonce.)

Expiry check. Each swap leg must be valid at the batch block number blk via

$$\text{blk} \leq \text{vto}(u),$$

with $\text{blk}, \text{vto}(u)$ range-checked in the chosen integer width.

Remark 9.1 (Public effects tuple $\text{pub}(u)$). We write $\text{pub}(u)$ for the ordered tuple of all public fields of a client bundle that the rollup circuit witnesses and uses to recompute $\text{inst}(u)$:

$$\text{pub}(u) = (\text{rt}(u), \text{nf}_1(u), \text{nf}_2(u), \text{cm}_1(u), \text{cm}_2(u), \text{st}(u), \text{sw}(u), \text{vto}(u)) \in \mathbb{F}^8.$$

Whenever the specification says “recompute $\text{inst}(u)$ from $\text{pub}(u)$ ”, it means to apply Definition 9.4 to the components of this tuple in exactly this order.

Definition 9.4 (Instance digest). Define the instance digest as a domain-separated hash of all public effects:

$$\text{inst}(u) := \text{Hash}(\text{pub}(u)) \in \mathbb{F}.$$

Remark 9.2 (Binding requirement). *The rollup must recompute $\text{inst}(u)$ from the witnessed $\text{pub}(u)$ and match it to the public input verified by the client-proof verifier gadget. This prevents mixing a valid proof with unrelated state updates. Since $\text{sw}(u)$ and $\text{vto}(u)$ are included in $\text{pub}(u)$, the recomputed $\text{inst}(u)$ also commits to them. Accordingly, the rollup verifier passes $\text{inst}(u)$ as the (single) client-proof public instance.*

9.4 Bundle-induced rollup state transition

Definition 9.5 (Apply (historic-root anchored)). *Let C be the commitment map, N the nullifier map, and H the historic-roots map. We write $\text{Apply}(u, C, N, H) = (C', N')$ iff:*

1. **Anchor is authorized:**

$$\text{get}(H, \text{rt}(u)) = 1.$$

2. **Insert output commitments into C :**

$$C' := \text{ins}(\text{ins}(C, \text{cm}_1(u), 1), \text{cm}_2(u), 1).$$

3. **Insert fresh nullifiers into N (sequential freshness):**

$$\text{get}(N, \text{nf}_1(u)) = 0, \quad N_1 := \text{ins}(N, \text{nf}_1(u), 1), \quad \text{get}(N_1, \text{nf}_2(u)) = 0,$$

$$N' := \text{ins}(N_1, \text{nf}_2(u), 1).$$

Definition 9.6 (ApplyPair (two transfers or a matched two-party swap)). *Let C be the commitment map, N the nullifier map, and H the historic-roots map. Let blk be the rollup batch L2 block number for this proof (bound in σ). We write $\text{ApplyPair}(u_L, u_R, C, N, H, \text{blk}) = (C', N')$ iff:*

1. **Either both are transfers:**

$$(\text{sw}(u_L), \text{st}(u_L), \text{vto}(u_L)) = (0, 0, 0) \wedge (\text{sw}(u_R), \text{st}(u_R), \text{vto}(u_R)) = (0, 0, 0),$$

2. **or both are swap legs with matching intent:**

$$\text{st}(u_L) = \text{st}(u_R) \neq 0,$$

$$\text{sw}(u_L) = \text{Hash}(\text{st}(u_L), \text{vto}(u_L)) \neq 0, \quad \text{sw}(u_R) = \text{Hash}(\text{st}(u_R), \text{vto}(u_R)) \neq 0,$$

$$\text{blk} \leq \text{vto}(u_L) \wedge \text{blk} \leq \text{vto}(u_R).$$

and in either allowed case the public state transition is

$$(C_1, N_1) = \text{Apply}(u_L, C, N, H), \quad (C', N') = \text{Apply}(u_R, C_1, N_1, H).$$

All other pairings are rejected.

Remark 9.3 (Swap pairing locality / adjacency requirement). *In this specification, two-party swaps are recognized only in the $\ell = 1$ leaf rollup step via $\text{ApplyPair}(u_L, u_R, \cdot)$. Therefore both swap legs must be placed as siblings in the recursion tree (i.e., paired together at layer 1). A batch builder / sequencer must enforce this ordering constraint or otherwise exclude unmatched swap legs from the batch.*

9.5 Aggregation state, instances, and messages

Definition 9.7 (Rollup aggregation state). *A rollup aggregation state is*

$$\sigma = (c_{\text{pre}}, c_{\text{post}}, n_{\text{pre}}, n_{\text{post}}, s, r_{\text{hist}}, \text{blk}) \in \mathbb{F}^7,$$

where $c_{\text{pre/post}} = \text{root}(C_{\text{pre/post}})$, $n_{\text{pre/post}} = \text{root}(N_{\text{pre/post}})$, $r_{\text{hist}} = \text{root}(H)$, blk is the (batch) L2 rollup block number, and s binds the ordered leaf tags in the subtree.

PCD-style public instance at layer $\ell \geq 1$.

$$x_\ell := (\sigma, \text{acc}, \ell),$$

where acc is the carried accumulator (encoded via $\text{PI}(\text{acc})$ in public inputs).

Messages. Messages carry only the node digest and the layer:

$$z := (s, \ell).$$

Thus children provide $z_L = (s_L, \ell - 1)$, $z_R = (s_R, \ell - 1)$.

9.6 Local compliance predicate φ_{rollup}

Layer 0 are client leaves; layer 1 aggregates two client bundles; layer > 1 merges two child rollup states.

Definition 9.8 (Local compliance predicate φ_{ROLLUP}). *On input outgoing message $z = (s, \ell)$, local data w , and incoming messages $z_L = (s_L, \ell - 1)$, $z_R = (s_R, \ell - 1)$ (or both $\perp$ when $\ell = 0$), accept iff:*

- **Base case $\ell = 0$:** $z_L = z_R = \perp$. (The client-proof system enforces layer-0 validity with public instance $\text{inst}(u)$; the emitted message is $z = (\text{LeafTag}(u), 0)$ where $\text{LeafTag}(u) = \text{inst}(u)$.)
- **Leaf rollup step $\ell = 1$:** parse w as

$$(u_L, u_R, C_{\text{pre}}, N_{\text{pre}}, H, \text{blk}, \pi_L, \pi_R, \omega_L, \omega_R, \rho_L, \rho_R, \pi_V),$$

and require:

1. **Incoming messages match client leaf tags:**

$$z_L = (\text{inst}(u_L), 0), \quad z_R = (\text{inst}(u_R), 0).$$

2. **Recompute instance digests from witnessed public effects:** First parse the witnessed public effects $\text{pub}(u_L), \text{pub}(u_R)$ of each bundle and require

$$\text{inst}(u_L) = \text{Hash}(\text{pub}(u_L)),$$

$$\text{inst}(u_R) = \text{Hash}(\text{pub}(u_R)).$$

3. **Bind historic-roots root:**

$$r_{\text{hist}} = \text{root}(H).$$

4. *Apply the pair as either two transfers or a matched swap pair:*

$$(C_2, N_2) = \text{ApplyPair}(u_L, u_R, C_{\text{pre}}, N_{\text{pre}}, H, \text{blk}).$$

5. *Expose semantic roots:*

$$c_{\text{pre}} = \text{root}(C_{\text{pre}}), \quad c_{\text{post}} = \text{root}(C_2), \quad n_{\text{pre}} = \text{root}(N_{\text{pre}}), \quad n_{\text{post}} = \text{root}(N_2).$$

6. *Bind batch block number into σ :*

blk is the subtree batch L2 rollup block number.

7. *Form subtree digest:*

$$s = \text{Hash}(\text{inst}(u_L), \text{inst}(u_R)).$$

8. *Verify the two client proofs (partial verifier):*

$$R_0^{\text{PA}}(\text{inst}(u_L), \pi_L; \omega_L, \rho_L) = 1, \quad R_0^{\text{PA}}(\text{inst}(u_R), \pi_R; \omega_R, \rho_R) = 1.$$

9. *Accumulator neutralization and fold: neutralize carried accumulators at $\ell = 1$ by $\text{acc}'_L = \text{acc}'_R := \text{ZeroAcc}$, and require*

$$\text{Verify}_{\text{AS}}(\text{avk}, [\rho_L, \rho_R], [\text{acc}'_L, \text{acc}'_R], \text{acc}, \pi_V) = 1.$$

• *Internal rollup step $\ell > 1$: parse w as*

$$(\sigma_L, \sigma_R, \pi_L, \pi_R, \text{acc}_L, \text{acc}_R, \omega_L, \omega_R, \rho_L, \rho_R, \pi_V),$$

where $\sigma_L = (c_{\text{pre}}^L, c_{\text{post}}^L, n_{\text{pre}}^L, n_{\text{post}}^L, s_L, r_{\text{hist}}^L, \text{blk}^L)$ and similarly σ_R . Require:

1. *Incoming messages match children:*

$$z_L = (s_L, \ell - 1), \quad z_R = (s_R, \ell - 1).$$

2. *Stitch boundary roots (deterministic composition):*

$$c_{\text{post}}^L = c_{\text{pre}}^R \quad \wedge \quad n_{\text{post}}^L = n_{\text{pre}}^R.$$

3. *Historic-root consistency:*

$$r_{\text{hist}}^L = r_{\text{hist}}^R.$$

4. *Batch block-number consistency:*

$$\text{blk}^L = \text{blk}^R.$$

5. *Form parent semantic state and digest:*

$$c_{\text{pre}} = c_{\text{pre}}^L, \quad c_{\text{post}} = c_{\text{post}}^R, \quad n_{\text{pre}} = n_{\text{pre}}^L, \quad n_{\text{post}} = n_{\text{post}}^R,$$

$$r_{\text{hist}} = r_{\text{hist}}^L, \quad \text{blk} = \text{blk}^L, \quad s = \text{Hash}(s_L, s_R).$$

6. *Verify child aggregation proofs (partial verifier):*

$$R_{\ell-1}^{\text{PA}}(x_{\ell-1}^L, \pi_L; \omega_L, \rho_L) = 1, \quad R_{\ell-1}^{\text{PA}}(x_{\ell-1}^R, \pi_R; \omega_R, \rho_R) = 1,$$

where $x_{\ell-1}^L = (\sigma_L, \text{acc}_L, \ell - 1)$ and $x_{\ell-1}^R = (\sigma_R, \text{acc}_R, \ell - 1)$.

7. *Accumulator fold:*

$$\text{Verify}_{\text{AS}}(\text{avk}, [\rho_L, \rho_R], [\text{acc}_L, \text{acc}_R], \text{acc}, \pi_V) = 1.$$

9.7 Historic-root update and terminal predicate ψ_{rollup}

Remark 9.4 (Anchor validity is checked against the pre-batch historic set). *In this construction the historic-roots root r_{hist} is constant throughout the aggregated subtree and is bound in the terminal predicate to the pre-batch historic map H_{pre} . Therefore every client anchor $\text{rt}(u)$ is validated against H_{pre} (not against roots created within the same batch). This intentionally disallows intra-batch “spend what was created earlier in the batch”.*

Definition 9.9 (Historic-root update with anti-replay). *Given $c_{\text{pre}}, c_{\text{post}} \in \mathbb{F}$ and a historic-roots map H_{pre} , define $H_{\text{post}} := \text{HistUpdate}(c_{\text{pre}}, c_{\text{post}}, H_{\text{pre}})$ by:*

$$\text{get}(H_{\text{pre}}, c_{\text{pre}}) = 1, \quad \text{get}(H_{\text{pre}}, c_{\text{post}}) = 0, \quad H_{\text{post}} = \text{ins}(H_{\text{pre}}, c_{\text{post}}, 1).$$

Definition 9.10 (Terminal merge predicate ψ_{ROLLUP} (FinalAggCircuit)). *Given top child states (σ_L, acc_L) , (σ_R, acc_R) at layer ℓ , a parent state (σ, acc) , a historic-roots pre-map H_{pre} , an expected post-root digest $r_{\text{post}} \in \mathbb{F}$, and a block-counter transition $(\text{blk}_{\text{pre}}, \text{blk}_{\text{post}}) \in \mathbb{F}^2$, accept iff:*

1. **Internal-step conditions hold** (stitching, $s = \text{Hash}(s_L, s_R)$, historic-root consistency, batch block-number consistency, parent formation) exactly as in the $\ell > 1$ branch of φ_{ROLLUP} .
2. **Bind historic-roots pre-root:** let $r_{\text{pre}} := \text{root}(H_{\text{pre}})$, and require

$$r_{\text{hist}} = r_{\text{pre}}.$$

3. **Historic-roots update:** compute $H_{\text{post}} := \text{HistUpdate}(c_{\text{pre}}, c_{\text{post}}, H_{\text{pre}})$, and require

$$\text{root}(H_{\text{post}}) = r_{\text{post}}.$$

4. **Global L2 block counter update:**

$$\text{blk}_{\text{post}} = \text{blk}_{\text{pre}} + 1.$$

5. **Bind subtree block number to the batch block number:**

$$\text{blk} = \text{blk}_{\text{post}}.$$

6. **Final decider handled outside:** the terminal circuit exposes $\text{PI}(\text{acc})$ as public input; a separate decider checks $\text{D}_{\mathcal{B}}(\text{acc}) = 1$.

FinalAggCircuit public inputs. Expose:

$$(\sigma, \text{acc}, \ell, \text{blk}_{\text{pre}}, \text{blk}_{\text{post}}, r_{\text{pre}}, r_{\text{post}}, \text{PI}(\text{acc})).$$

Concretely:

$$(c_{\text{pre}}, c_{\text{post}}, n_{\text{pre}}, n_{\text{post}}, s, r_{\text{hist}}, \text{blk}, \ell, \text{blk}_{\text{pre}}, \text{blk}_{\text{post}}, r_{\text{pre}}, r_{\text{post}}, \text{PI}(\text{acc})).$$

Definition 9.11 (On-chain state and head-pinning validity rule). *Let the on-chain state be*

$$\text{St} = (\text{blk}_{\text{head}}, c_{\text{head}}, n_{\text{head}}, r_{\text{hist}}, r_{\text{DA}}).$$

A proof output $(\sigma, \text{acc}, \ell, \text{blk}_{\text{pre}}, \text{blk}_{\text{post}}, r_{\text{pre}}, r_{\text{post}})$ is admissible for state update only if its pre-roots equal the current on-chain heads:

$$\text{blk}_{\text{pre}} = \text{blk}_{\text{head}}, \quad c_{\text{pre}} = c_{\text{head}}, \quad n_{\text{pre}} = n_{\text{head}}, \quad r_{\text{hist}} = r_{\text{pre}}.$$

Upon acceptance, the contract updates:

$$\text{blk}_{\text{head}} \leftarrow \text{blk}_{\text{post}}, \quad c_{\text{head}} \leftarrow c_{\text{post}}, \quad n_{\text{head}} \leftarrow n_{\text{post}}, \quad r_{\text{hist}} \leftarrow r_{\text{post}},$$

(and updates/records the DA commitment for the posted batch).

Remark 9.5. Membership in H alone does not prevent forks: the L1 contract must pin c_{pre} and n_{pre} to the current heads.

References

- [BCMS] B. Bünz, A. Chiesa, P. Mishra and N. Spooner, *Proof-Carrying Data from Accumulation Schemes*, IACR Cryptology ePrint Archive, 2020:499, 2020.
- [GWC] A. Gabizon, Z. J. Williamson and O. Ciobotaru, *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*, IACR Cryptology ePrint Archive, 2019:953, 2019.